```
      const schedule = document.getElementById(target.dataset.scheduleId)
      schedule.classList.toggle("hidden", false)
    })
  }
}
```

The filter action is the more complicated of the two actions. We start by using the DOM data attributes to determine the state of the system in the everyDayUnselected method. The everyDayUnselected method returns true if and only if the cssStatusValue for every one of the targets is "false". Unfortunately, we have to check against the string version of "false" because the value is part of a different controller; otherwise, we'd be able to use the Stimulus Values API to define the value as a Boolean and have Stimulus typecast it for us.

For each of our calendarDayTargets, meaning for each of the date displays, we determine if the corresponding schedule item should be shown. The answer is yes if the cssStatusValue is "true" or if every day is unselected. Then we use the regular DOM getElementById method to determine our matching schedule element and set a hidden class on that element. This works because we know that the click->css#toggle action has already happened, so we know that the cssStatusValue has already changed to its new value.

The showAll method similarly loops over all our calendar day targets. But instead of using the filter logic, it just sets the cssStatusValue of all of them to false. Doing so triggers the cssStatusValueChanged method of the CssController, which removes the border class we set. It doesn't also automatically reset the corresponding schedule item, so we need to do that manually.

This works, showing how we can use the DOM to store the data we need for more complex client-side effects.